

An Unsupervised TinyML Approach with Efficient Edge AI for Effective Stress and Sleep Monitoring

M Sudha¹, P. Neelaveni², E Pooja³, V Mangaiyarkarasi⁴, Rowsonara Begum⁵, J. Jayanthi⁶

¹Assistant professor, Department of Electronics and Communication Engineering, Adhi College of Engineering and Technology, Kancheepuram, Tamil Nadu, India.

²Department of Information Technology, Rajalakshmi Engineering College, Chennai

³Assistant Professor, Department of Computer Science and Engineering, Rajalakshmi Institute of Technology, Chennai, Tamil Nadu, India.

⁴Associate Professor, Department of ECE, New Prince Shri Bhavani College of Engineering and Technology, Chennai, Tamil Nadu, India.

⁵Assistant Professor, Department of Computer Science & Engineering (Data Science), MLR Institute of Technology, Dundigal, Hyderabad, Telengana -500043

⁶Professor, Department of Computer Science and Engineering, Sona College of Technology, Salem

Email id: msudhani2802@gmail.com, neelaveni.p@rajalakshmi.edu.in, poojae238@gmail.com, mangai.manikandan@gmail.com, rowshanarabegum45@gmail.com, jayanthij@sonatech.ac.in

Abstract- The creation of embedded technologies for detecting stress and categorizing sleep stages has been prompted by the need for monitoring health in real time. The UTMESS model combines data from various sensors, deep learning, and AI to monitor health using affordable devices. It improves the representation of features for accurate modeling of physiological states by collecting data from inertial measurement units and heart rate variability to create 12-channel tensors. The proposed model uses transfer learning with MobileNetV2 for health classification and an LSTM autoencoder for the effective monitoring of stress and sleep process. It is optimized for embedded devices through pruning and feature engineering. When run on the ESP32S3 microcontroller, UTMESS achieves 98.5% accuracy in the simulation demonstration which effectively distinguishes stress levels according to the given data, aligns well with known sleep stages, uses low energy, and performs excellently.

Keywords— *TinyML Process, Unsupervised Learning Approach, Edge Artificial Intelligence Model and Effective Stress and Sleep Monitoring.*

I. INTRODUCTION

Machine Learning (ML) plays a significant role in realizing the potential of the internet-based environment by allowing devices to make proper decisions, predictions, and automation which results in the reduction of man power [1]. Traditional ML typically relies on cloud network, which sends the information from various devices which are connected to internet and to central servers. While

effective, this approach faces issues like transmission delay and lack of security. To address these issues, TinyML has been developed [2]. TinyML focuses on using ML algorithms on small, low-power devices that do not require extensive computing power or cloud services. The need for smart devices has driven the popularity of TinyML, enabling real-time processing without relying on cloud resources which can be cost-prohibitive and pose security risks [3]. TinyML is specifically designed for IoT devices and low-power processors, allowing for efficient processing at the edge of networks. This approach results in advantages such as improved privacy, less bandwidth usage, and increased autonomy for devices in remote locations [4]. With the rapid growth of connected devices, expected to increase from 15 billion in 2023 to 29 billion by 2030, the volume of data generated from IoT is also rising significantly [5].

The proposed UTMESS model combines IMU and HRV data to create 12-channel tensors from raw sensor signals. It enhances signal quality and strength on devices using special preprocessing methods like signal normalization and transformations tailored for the LSM9DS1 sensor package. UTMESS employs a transfer learning approach with MobileNetV2 for classifying physiological data and uses LSTM autoencoders to detect time-based anomalies. It can run efficiently on edge devices using TensorFlow Lite and embedded

C, making real-time execution feasible on ESP32S3 microcontrollers. UTMESS also features a modular processing pipeline for various biomedical tasks, ensuring reusability and maintainability. The organization of the paper is divided into sections below. Section 2 discusses previous models and flaws. Section 3 develops a new UTMESS model in detail. Section 4 is an experimental analysis of the UTMESS model. Section 5 explains the overall proposed conclusion.

II. RELATED WORKS

In [6], a AIoT uses LSTM autoencoders to identify point and collective anomalies in urban noise data, achieving high accuracy rates of 99.99% and 99.34%. The models run well on ESP32 MCUs with quick inference times and low resource usage. In [7], Open-source C library for edge device training and inference. It fills a need in edge AI by enabling on-device learning and has been tested for performance and power use on popular microcontrollers. In [8], a low-cost, open-source tool that uses vibration analysis to find bearing faults and teach embedded AI. It combines mechanical, software, and hardware elements through platforms like Arduino to enhance STEM learning. In [9], using intracranial EEG signals, a TinyML model can identify seizures in real time with 98–99% precision. It requires little computing power, making it suitable for small, implantable neurostimulation systems that identify seizures. In [10], a CNN-based system for CubeSats analyses RGB images to identify cloud coverage, helping prioritize image transmission and optimize limited communication bandwidth. It works effectively on a common microcontroller, achieving 90% accuracy on test datasets, supporting self-sufficient Earth observation satellites. In [11], a technology enables ML and DL approaches to run on low-resource IoT devices without cloud processing. The study of TinyML research, discussing model types, datasets, and device characteristics. In [12], an affordable, wearable device uses IoT, edge computing, and TinyML to monitor walking patterns in real time. It employs an IMU sensor and machine learning for early diagnosis and therapy outside clinics. In [13], a CNN architecture designed for TinyML that is efficient in memory use. It uses techniques such as a stream-based line-buffer design and accelerator-aware pruning. These strategies allow models like MobileNetV1 and MobileNetV2 to be stored on-

chip, eliminating the need for external memory. In [14], edge AI uses TinyML algorithms like TinyCleanEDF and EdgeClusterML to improve data management in large IoT systems. On Raspberry Pi devices, algorithms work well together to clean and compress data, showing great effectiveness in performance. In [15], A health monitoring system uses TensorFlow Lite and TinyML on microcontrollers to study stress and sleep by analyzing HRV data. It shows that TinyML can provide personalized health information, achieving accuracy rates of 90% for younger users. In [16], a lightweight AI method uses signal preprocessing and ultrasonic guided waves to find structural faults with over 90% accuracy for TinyML use. It includes cloud-based heatmap visualizations, reducing computing needs and enabling real-time analysis on cheap devices. In [17], a pruning-based approach helps reduce the size of Transformer models like MobileBERT, TinyViT, and tinyLLAMA for use in TinyML applications on low-power RISC-V microcontrollers. This method cuts memory by up to 94% and boosts inference speed to 1219 tokens/s while consuming only 57 mW of power. In [18], a MobileNetV2 model using TinyML is created for quick disease detection in apple and mango leaves, achieving 93.1% accuracy, which is better than traditional models like ResNet-50 and VGGNet-16. It is suitable for edge deployment in resource-limited agriculture but currently only detects a few diseases and crops. In [19], modern ways to use AI on low-resource machines, focusing on model compression, TinyML frameworks, and federated learning for efficient and private edge intelligence. It shows applications in healthcare, agriculture, and smart cities while providing some solutions to current challenges. In [20], advanced feature engineering methods in TinyML-based embedded systems for helping people with hand rehabilitation after a stroke. These methods improve gesture recognition accuracy and reduce costs by up to 31%. The system uses techniques like MRMR, t-tests, and random forest selection for real-time robotic therapy.

III. PROPOSED METHODOLOGY

The UTMEES model is designed to predict sleep and stress accurately by using deep learning, embedded systems, and multi-sensor data. It first collects heart rate variability (HRV) and IMU data from two datasets. This data is then normalized and split into smaller segments. Sensors provide

accelerometer, gyroscope, and orientation data, which are transformed into 12-channel tensors for better feature representation. Depending on the platform, either neural networks or bagged forests process the data. The model runs in real-time on a ESP32S3, using an LSTM autoencoder to detect abnormalities and a MobileNetV2 for health condition classification. The model is designed for efficient on-device inference with TensorFlow Lite. Figure 1 shows the structure of UTMEDD process in detail.

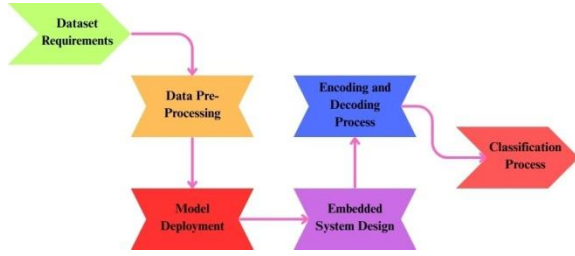


Figure 1 – Structure of UTMEDD Process

Dataset Requirements:

The ISRUC and SWELL datasets were used to predict stress and sleep in this study. They supplied HRV data with over 30 markers, which include time-domain measurements such as SDNN, RMSSD, and pNN50, as well as frequency-domain features like LF, HF, and LF-HF ratios. The data was normalized for sensor differences, segmented into epochs for specific activities like sleep and stress, and cleaned by removing noise and artifacts through filtering methods.

Data Pre-Processing:

Data preprocessing is needed because of the sensor's limitations. Sensor data is normalized based on the sensor's total limits. The accelerometer data point is normalized.

$$a_n = a/8 = a/2, \quad (1)$$

The LSM9DS1 sensor detects values from -4g to 4g in x, y, and z directions, normalizing gyroscope values.

$$g_n = g2000/4000 = g/2 \quad (2)$$

The gyroscope sensor readings for x, y, and z go from -2000 to 2000 degrees per second. The data is averaged before being fed into a deep learning network. The LSM9DS1 is an IMU that includes a 3-axis accelerometer, magnetometer, and gyroscope, with a maximum sampling rate of 104 Hz. To enhance feature extraction for the CNN model, data

is augmented with orientation angle, accelerometer, and gyroscope information. This creates a final data vector of size $N \times 128 \times 12$.

$$R_a = \sqrt{A_x^2 + A_y^2 + A_z^2}, \quad (3)$$

$$R_g = \sqrt{G_x^2 + G_y^2 + G_z^2}, \quad (4)$$

$$R_o = \sqrt{O_\theta^2 + O_\phi^2 + O_\psi^2}, \quad (5)$$

The resulting vector for acceleration, gyroscope, and orientation is represented by R_a , R_g , and R_o . A data tensor is created from $N \times 128 \times 9 \times 1$ to $N \times 128 \times 12 \times 1$ using specific equations (3) and (5). This novel tensor includes 12 channels, which helps improve the machine learning model by allowing it to learn complex patterns and perform better on new data.

Model Deployment

Feature engineering results are important for training and implementing models. Neural networks and bagged forests are deployed differently. For neural networks, they are exported using the edge impulse framework without needing transformations like pruning or quantization. The cost complexity parameter, ccp-alpha, is set to 0.001 to prune the bagged forest. The Everywhere (v0.2.40) Python library helps translate each estimator into C code by turning thresholds and bifurcations into if-else statements. A sampling method with a 95% confidence interval and a 3% margin of error assessed the AI model's predictive accuracy. A conservative p value of 0.5 and a Z-score of 1.96 were used for better variance estimation. The sample size was adjusted to about 1008 observations from 17,520 samples to ensure equal representation across six classification groups for comparing model performance on embedded systems and conventional computers.

Embedded System Design

The architecture includes a processing pipeline with connected parts. It starts with collecting raw surface electromyography (sEMG) signals and finishes with recognizing the gesture. This pipeline has eight modules, each handling specific functions, which promotes modularity, code reusability, and easy maintenance. These modules work together to transform raw bio-signals into categories for real-time gesture recognition. The model uses the XIAO ESP32S3 module, featuring a 32-bit dual-core CPU. It operates with an interrupt-assisted pooling cycle at

500 Hz to manage timer tasks and read analog inputs without losing data. The EMG Sampling block saves samples in a Circular Buffer, while the Timer block keeps track of time for the Segmentation and Display blocks. The Segmentation block generates 200-millisecond windows for model predictions, processing the last 100 data points without overlap. Features are stored in the Display block, which manages user input and filters data before sending it to the model.

Encoding and Decoding Process:

The encoder generates a low-dimensional encoded feature vector at the bottleneck after receiving input data. From this feature, the decoder then restores the original input sequence. The reconstruction error is the gap between the decoder's output and the original input. An LSTM autoencoder was employed to find inconsistencies in urban noise data from a sensor network. LSTM, a kind of RNN, is adept at retaining inputs over extended periods, making it useful for analyzing sequential and time series data. The forget gate, the input gate, and the output gate are the three gates that make up a standard LSTM network. Depending on the prior hidden state h_{t-1} , the forget gate employs a sigmoid function to choose which data to forget. The input gate (Eq. (6)) uses a tanh function to process new input data X_t and the previous hidden state h_{t-1} in order to determine what new data X_t to remember. The results from each gate are combined to determine the new cell state. The new hidden state h_{t-1} is managed by the output gate, which uses a sigmoid function on the new input X_t and the previous hidden state h_{t-1} and a tanh function on the updated cell state C_t to generate the new hidden state.

$$i_t = \sigma(W_i^X X_t + W_i^h h_{t-1} + b_i) \quad (6)$$

$$f_t = \sigma(W_f^X X_t + W_f^h h_{t-1} + b_f) \quad (7)$$

$$o_t = \sigma(W_o^X X_t + W_o^h h_{t-1} + b_o) \quad (8)$$

$$C_t = f_t \circ C_{t-1} + i_t \tanh(W_c^X X_t + W_c^h h_{t-1} + b_c) \quad (9)$$

$$h_t = o_t \tanh(C_t) \quad (10)$$

The cell state is C_t , the input data is X_t , and the previous hidden output is h_{t-1} . The gates include the input gate i_t , the forget gate f_t , and the output gate o_t . The weight matrix is W , and the Hadamard product is $\circ$.

Classification Process:

The MobileNetV2 was adapted through transfer learning model which is mainly used to identify plant diseases. A pooling layer was utilized to reduce data, followed by a new layer to determine if leaves are healthy or diseased. MobileNetV2 is effective for TinyML applications due to its balance of accuracy and efficiency, suitable for low-resource devices. Testing was performed on a Windows 11 MSI device with 16 GB of RAM, using a pretrained MobileNetV2 base as a feature extractor. The model's base was fine-tuned and a Global Pooling layer was added, along with a connected layer which gets activated by ReLU. To prevent overfitting, a Dropout layer with a 50% was included. Data augmentation techniques, such as random rotations and flips, were applied to diversify the dataset. The model trained for 30 epochs at a learning rate of 0.0001, using the Adam optimizer and categorical cross-entropy loss. Ultimately, the model was converted to TensorFlow Lite format for low-power edge devices, and the UTMESS model combines advanced signal processing, deep learning, and edge computing for efficient health monitoring. The system uses both traditional and neural models for classification and accurately extracts features from multiple sensors. Its modular design enables real-time gesture recognition and reduces data loss. It employs LSTM to categorize stress and sleep states and MobileNetV2 for classification. TensorFlow Lite allows it to work on low-power devices without losing accuracy. UTMESS is reliable and effectively uses resources, making it suitable for remote health monitoring and wearables.

IV. RESULTS AND DISCUSSION

Model Accuracy of UTMESS:

The UTMESS model's training accuracy improves from 94.2% to 98.15%, and its validation accuracy rises from 95.5% to 98.5%, showing effective learning and strong generalization. The training and validation curves align well, indicating minimal overfitting. Unlike basic models like MobileNet and Random Forests, UTMESS provides higher accuracy and stability on embedded devices. TensorFlow Lite facilitates its lightweight deployment and data integration, and its LSTM autoencoders excel in managing time-series irregularities. The MobileNetV2-based classifier benefits from transfer learning, enhancing robustness in noisy settings, while dropout and augmentation

reduce overfitting. The model's design assures good performance on edge devices, ensuring reliability under constraints. The reported accuracy of up to 98.5% is supported by a confidence interval assessment.

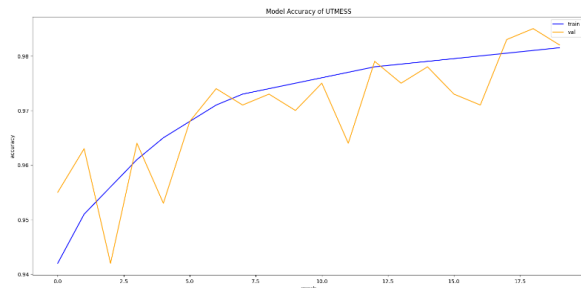


Figure 2 - Model Accuracy of UTMESS

Different Stress Levels of UTMESS:

Figure 3 shows that the UTMESS, which categorizes data into three classes: interrupt, no stress, and stress. It shows four subplots representing the model's predictions. In subplot (a), the model is very certain about "no stress," with a probability of 1.0. Subplot (b) indicates moderate confidence in "stress" at 60%, showing some overlap among classes. Subplot (c) has a strong focus on "stress," with 89% probability, indicating a precise forecast due to good feature representation. However, subplot (d) shows a misclassification where the model strongly predicts "interrupt" (95%), which may not reflect the actual situation, indicating possible false positives or class imbalance. It uses LSTM autoencoders for better temporal learning, which helps to clearly differentiate between "stress" and "no stress." Simpler models like naive Bayes or logistic regression struggle with overlapping probabilities, resulting in lower performance. UTMESS avoids accuracy loss common in edge deployments by not using quantization.

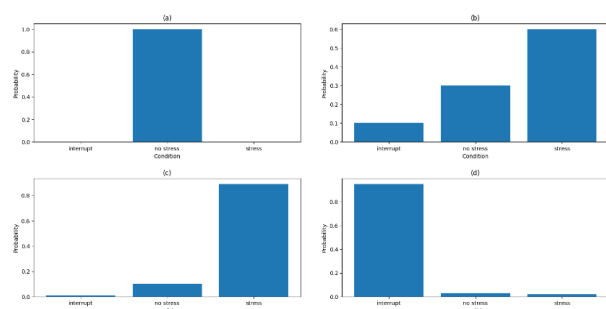


Figure 3 - Different Stress Levels of UTMESS

Heart Rate Vs Sleep Stage of UTMESS:

Figure 4 shows that the UTMESS model studies

heart rate during five sleep phases: W, N1, N2, N3, and R. Each phase has a different heart rate pattern, with the most variability in the waking phase (W), showing irregular heart activity linked to movement or awareness. Deeper sleep stages, N2 and N3, show steadier heart rates. The heart rate decreases from W to N2, then slightly rises in N3 and R, which aligns with known sleep physiology. UTMESS uses information from multiple sensors instead of just time-domain features to better analyze heart rate variability. It improves the detection of sleep stages and adapts to noisy signals by employing DL approaches like CNNs and LSTMs. TensorFlow Lite allows UTMESS to maintain accuracy while using less processing power, making it very much suitable for real-time environment.



Figure 4 - Heart Rate Vs Sleep Stage of UTMESS

V.CONCLUSION

The proposed UTMESS is mainly designed to monitor the physiological states in real time through the presence of edge computing, multi-sensor fusion, and deep learning. It uses IMU and HRV data from ISRUC and SWELL datasets, to improve the feature extraction. The proposed UTMESS has a dual architecture featuring a MobileNetV2 classifier for sleep and stress detection and an LSTM autoencoder for finding abnormalities. It handles both sequential and static signals in an effective manner. Techniques like data augmentation and transfer learning boost its performance and prevent overfitting, making it suitable for microcontroller applications. The system's modular setup optimizes memory and reduces latency in sensor data processing for health monitoring. The LSTM module accurately reconstructs time series patterns. Finally this proposed UTMESS model is portable, secure, and energy-efficient due to TensorFlow Lite, effectively linking heart rate and motion data to monitor sleep and stress.

REFERENCES

- [1]. A. Ramadan, et.al, "FL and TinyML on Internet of Things edge devices: Challenges, advances, and upcoming research scope", *ICT Express*, 2025, DOI: 10.1016/j.icte.2025.06.008
- [2]. M. R. Al-Mousa et al, "ENOP: A Robust Methodology to Predict Working Employee Stress Levels Using Enhanced Neural Optimization Principle," (ICSES), Chennai, India, 2024, pp. 1-6, doi: 10.1109/ICSES63760.2024.10910842.
- [3]. M. Carcelos, et.al, "MicroFlow: An Efficient Rust-Based Inference Engine for TinyML", *IoT*, vol. 30, 2025, DOI: /10.1016/j.iot.2025.101498
- [4]. G. R et al, "A Novel Development of Artificial Intelligence Enabled Learning Methodology for Human Depression Prediction Scheme," (ISCS), Gurugram, India, 2024, pp. 1-6, doi: 10.1109/ISCS61804.2024.10581026.
- [5]. M. Ficco. et.al, "FL for Internet of Things devices: Improved TinyML with on-board training", *Information Fusion*, vol. 104, 2024, DOI: 10.1016/j.inffus.2023.102189
- [6]. M. O. Sabri et al, "Empirical Possibilities of Social Network Privacy and Security Enhancement to Improve Data Communication Efficiency," (ICETEMS), Nagpur, India, 2024, pp. 650-655, doi: 10.1109/ICETEMS64039.2024.10964976.
- [7]. A. Karavaev, et.al " TinyDecisionTreeClassifier: Embedded C++ library for training and applying decision trees on the edge", *SoftwareX*, vol. 27, 2024, DOI: 10.1016/j.softx.2024.101778
- [8]. A. Herrera, et.al, "Low-cost prototype for bearing failure diagnosis through TinyML through vibration analysis", *HardwareX*, vol. 22, 2025, DOI: 10.1016/j.ohx.2025.e00658
- [9]. E. Tsakanika, et.al, "High Accuracy of Epileptic Seizure Diagnosis Through TinyML for Implantable Closed-Loop Neurostimulation Systems", vol. 5, 2025, DOI: 10.3390/biomedinformatics5010014
- [10]. C. Salazar, et.al, "Cloud Detection Autonomous System Based on ML and COTS Components On-Board Small Satellites", vol. 14, 2022, DOI: 10.3390/rs14215597
- [11]. N. N. Alajlan, et.al, "TinyML: Enabling of Inference DL Approaches on Ultra-Low-Power Internet of Things Edge Devices for Artificial Intelligence Applications", vol. 13, 2022, DOI: 10.3390/mi13060851
- [12]. S. Shakya, et.al, "CNN-Based Low-Powered Wearable Smart Device for Gait Abnormality Detection", vol.4, 2023, DOI: 10.3390/iot4020004
- [13]. H. Kang, et.al, "AoCStream: All-on-Chip Convolutional Neural Network Accelerator with Stream-Based Line-Buffer Framework and Accelerator-Aware Pruning", vol. 23, 2023, DOI: 10.3390/s23198104
- [14]. A. Karras, et.al, "Tiny Machine Learning Model for Big Data Management in Large-Scale IoT Architecture", vol. 16, 2024, DOI: 10.3390/fi16020042
- [15]. P. Srivastava, et.al, "Microcontroller-Based EdgeML: Health Monitoring for Stress and Sleep via HRV", vol. 78, 2024, DOI: 10.3390/engproc2024078003
- [16]. J. Henkmann, et.al, "Tiny ML Demonstration for Guided Wave-Based Damage Localization", vol. 25, 2025, DOI: 10.3390/s25020578
- [17]. A. Dequino, et.al, "Optimizing BFloat16 Localization of Tiny Transformers on ULP Extreme Edge SoCs", vol. 15, 2025, DOI: 10.3390/jlpea15010008
- [18]. I. Attri, et.al, "Tiny Machine Learning for Plant Disease Diagnosis: An Effective Edge Artificial Intelligence Solutions for Apple and Mango Leaves", vol. 258, pp. 2870-2877, 2025, DOI: 10.1016/j.procs.2025.04.547
- [19]. L. Pazmiño Ortiz, "Advancing TinyML in Internet of Things: A Holistic System-Level Perspective for Resource-Constrained AI", vol. 17, 2025, DOI: 10.3390/fi1706025
- [20]. Andres G, et. al, "On the Deployment of Edge Artificial Intelligence Models for Surface Electromyography-Based Hand Gesture Recognition", vol. 6, 2025, DOI: 10.3390/ai6060107